When Does Merkle-Based Authentication Outgrow X.509? Measured Certificate-Size Crossovers in Classical and Post-Quantum TLS

Author 1, Author 2, Author 3

***Abstract*—Post-quantum signatures substantially increase the authentication material carried by TLS, particularly when certificate paths contain multiple transmitted certificates.**

Merkle Tree Certificates (MTCs) provide a different scaling regime: the credential contribution grows mainly with the Merkle inclusion proof rather than by repeating complete public-key certificates.

This study asks when an MTC-based TLS `Certificate` message ceases to be smaller than its X.509 counterpart.

We evaluate nine signature profiles—ECDSA P-256, P-384, and P-521; ML-DSA-44, -65, and -87; and SLH-DSA-SHA2-128s, -192s, and -256s—against three X.509 trust-path profiles.

The final server-only crossover campaign contains 59 MTC measurement cells, with 30 measured TLS 1.3 handshakes per cell, and uses targeted intermediate tree sizes around the ECDSA transition regions.

Rather than extrapolating an exact crossover beyond the implemented range, we report the last tested population for which MTC remains smaller and the next tested population for which it does not.

For ECDSA, the raw crossover intervals are $(4, 8)$, $(8, 16)$, and $(16, 32)$ for P-256, P-384, and P-521 under the direct-server baseline, and $(8, 16)$, $(16, 32)$, and $(64, 128)$ under the CA-server baseline.

With a transmitted intermediate certificate, all three ECDSA profiles remain smaller through $N = 1024$.

All evaluated ML-DSA and SLH-DSA profiles likewise remain smaller than X.509 through $N = 1024$ for every tested chain profile.

A thresholded sensitivity analysis further shows why a raw byte crossover can be misleading: for P-521/CA-server, MTC is still smaller at $N = 64$, but only by one byte (0.169%); requiring a 1% or 5% saving moves the effective crossover to $(32, 64)$, and a 10% saving moves it to $(16, 32)$.

Complementary experiments show that these representation savings should not be interpreted as an unconditional latency advantage.

A fixed-$N = 16$ mutual-authentication campaign also shows that bidirectional PQ X.509 authentication increases median transport volume from 2,029 to 18,767 bytes ($9.249\times$), whereas the evaluated MTC combinations remain within 35 bytes of the classical X.509/X.509 baseline.

A dedicated mutual-authentication crossover campaign then extends the certificate-size metric bidirectionally by summing the server and client TLS `Certificate` messages. Across 23 cases, 17 MTC cells, and 690 measured handshakes, the raw direct-baseline intervals are $(1, 4)$, $(4, 8)$, and $(16, 32)$ for P-256, P-384, and P-521, while the CA-baseline intervals are $(8, 16)$, $(16, 32)$, and $(32, 64)$. The thresholded view again exposes marginal regions: for P-521/direct, the raw interval remains $(16, 32)$, but the 1% and 5% effective intervals move to $(8, 16)$ and the 10% interval to $(4, 8)$.

***Index Terms*—TLS 1.3, X.509, Merkle Tree Certificates, post-quantum cryptography, ML-DSA, SLH-DSA, certificate size, authentication, PKI scalability.**

509X.509

Author names, affiliations, acknowledgments, and venue-specific metadata to be completed before submission.

I. Why Authentication Size Matters Again

TLS authentication has long treated certificate-chain overhead as a manageable fixed cost.

That assumption becomes less comfortable in a post-quantum setting.

Large public keys and signatures can make a single certificate several kilobytes long, and every additional transmitted intermediate certificate repeats that cost.

The effect is especially visible for hash-based signatures such as SLH-DSA, but it is also relevant for ML-DSA.

Merkle Tree Certificates (MTC) explore a different design point.

Instead of representing authentication as a conventional chain of independently signed X.509 certificates, an MTC credential proves membership in an authenticated Merkle structure.

The important scaling difference is immediate:

an X.509 chain grows approximately with the number and size of transmitted certificates, whereas a Merkle proof grows logarithmically with the credential population.

This leads to a question that is easy to state but has not been sufficiently quantified:

> *For a given signature family and PKI depth, how large can the Merkle population become before the TLS Certificate message becomes larger than its X.509 counterpart?*

The answer matters for two reasons.

First, it tells us whether Merkle-based authentication remains attractive only for small deployments or also at Internet-scale populations.

Second, it reveals how the answer changes once classical signatures are replaced by post-quantum signatures.

This study focuses deliberately on *certificate representation size*.

It does not claim that a smaller TLS Certificate message always means a faster complete handshake.

CertificateVerify, record framing, CPU cost, and network behavior are separate dimensions and should be evaluated separately.

II. What We Compare

We evaluate nine authentication profiles grouped into three families:

- ECDSA P-256, P-384, and P-521;
- ML-DSA-44, ML-DSA-65, and ML-DSA-87;
- SLH-DSA-SHA2-128s, -192s, and -256s.

The server-only crossover uses three X.509 trust-path profiles.

The *direct-server* profile transmits only the server certificate and trusts that credential directly.

The *ca-server* profile also transmits one server certificate, but validation terminates at a locally configured CA trust anchor.

The *root-intermediate-server* profile transmits the server certificate plus one intermediate CA certificate, while the root remains local.

These profiles deliberately separate certificate representation from trust-anchor distribution: local trust material is not counted as TLS traffic.

The size boundary is the complete TLS `Certificate` handshake message, including the handshake header and body. `CertificateVerify`, TLS record headers, latency, and CPU time are excluded from the crossover metric.

For MTC, the measured message contains the experimental landmark credential and the implementation-specific confirmation material required by the evaluated TLS integration.

Pre-distributed MTC trust material is likewise excluded from per-connection TLS bytes and is treated as a deployment prerequisite rather than as zero-cost data.

The final crossover campaign evaluates profile-specific tree sizes.

ECDSA P-256 uses $N \in \{1, 4, 8, 16, 64, 256, 1024\}$; P-384 uses $N \in \{1, 4, 8, 16, 32, 64, 256, 1024\}$; and P-521 uses $N \in \{1, 4, 16, 32, 64, 128, 256, 1024\}$.

Each ML-DSA and SLH-DSA profile uses $N \in \{1, 4, 16, 64, 256, 1024\}$.

The targeted ECDSA points were added only where the coarse sweep bracketed a transition.

In total, the unified campaign contains 59 MTC cells.

Each cell uses five warmups followed by 30 measured handshakes, for 1,770 measured MTC handshakes in the final crossover run.

All measurements use TLS 1.3, X25519 key exchange, TLS_AES_256_GCM_SHA384, server-only authentication, no tickets, and no session resumption.

The MTC evaluation targets `draft-ietf-plants-merkle-tree-certs-05` together with the trust-anchor-identifier dependency.

The implementation should therefore be understood as an experimental, draft-targeted realization rather than a claim of final-standard interoperability.

III. Measured Crossover Methodology

The crossover analysis is based on measured TLS messages rather than on an extrapolated maximum-population model.

For each cryptographic profile p and X.509 chain profile c, let

$B_{\mathrm{X509}}(p, c)$ denote the median number of bytes in the observed TLS 1.3 `Certificate` message.

For the corresponding MTC landmark credential at tree size N, let

$B_{\mathrm{MTC}}(p, N)$ denote the measured median.

We define the absolute byte advantage as

$$\Delta B(p, c, N) = B_{\mathrm{X509}}(p, c) - B_{\mathrm{MTC}}(p, N), \tag{1}$$

and the relative saving as

$$G(p, c, N) = 100 \frac{\Delta B(p, c, N)}{B_{\mathrm{X509}}(p, c)}. \tag{2}$$

Positive values indicate that MTC transmits fewer `Certificate`-message bytes than X.509.

Because the implemented campaign samples discrete tree sizes, we do not report an unobserved exact crossover.

Instead, the raw crossover N^* is bracketed by two tested populations:

the largest tested $N = a$ for which $\Delta B > 0$, and the next tested $N = b$ for which $\Delta B \leq 0$.

The result is reported as the interval (a, b).

If MTC remains smaller at the largest implemented population, the result is reported as > 1024.

This interval notation makes the evidentiary boundary explicit and avoids converting a coarse sweep into an unjustified point estimate.

A one-byte advantage is technically a crossover but may not be operationally meaningful.

We therefore also define an effective crossover N^*_τ for a minimum relative saving threshold τ:

$$G(p, c, N) \geq \tau, \tag{3}$$

with $\tau \in \{1\%, 5\%, 10\%\}$.

The same bracketing rule is applied to the largest tested population meeting the threshold and the next tested population failing it.

This sensitivity analysis distinguishes a mathematical byte advantage from a practically material one.

The measured inclusion-proof contribution follows the expected logarithmic pattern for the tested power-of-two populations:

0 bytes at $N = 1$, 32 at $N = 4$, 64 at $N = 8$, 96 at $N = 16$, 128 at $N = 32$, 160 at $N = 64$, 192 at $N = 128$, 224 at $N = 256$, and 288 at $N = 1024$.

The TLS `Certificate`-message size therefore grows in small steps as the Merkle path deepens, while the X.509 baseline is fixed for a given cryptographic and chain profile.

IV. The Crossover Map

Table I reports the measured server-only crossover intervals.

The contrast between classical and post-quantum profiles is immediate.

ECDSA crosses inside the implemented range for the two one-certificate X.509 baselines, whereas every root-intermediate ECDSA case remains favorable through $N = 1024$.

All evaluated ML-DSA and SLH-DSA profiles also remain favorable through $N = 1024$ for every tested chain profile.

For the direct-server baseline, the ECDSA transition moves monotonically with key size: P-256 crosses between $N = 4$ and $N = 8$, P-384 between $N = 8$ and $N = 16$, and P-521 between $N = 16$ and $N = 32$.

The CA-server baseline shifts the corresponding intervals to $(8, 16)$, $(16, 32)$, and $(64, 128)$.

Once an intermediate certificate is transmitted, the X.509 baseline grows enough that all three ECDSA MTC configurations remain smaller through the largest implemented population.

A. Effective crossover under minimum-saving thresholds

The raw crossover alone can overstate the practical value of a marginal byte advantage.

Table II therefore reports the effective crossover N^*_τ for 1%, 5%, and 10% minimum savings.

Post-quantum profiles are omitted from the table because every ML-DSA and SLH-DSA configuration remains above the 10% saving threshold through $N = 1024$.

The P-521/CA-server case is the clearest example of why the thresholded view is necessary.

At $N = 64$, the X.509 `Certificate` message is 593 bytes and MTC is 592 bytes.

The raw criterion therefore still labels MTC favorable, but the gain is only one byte, or 0.169%.

Requiring merely a 1% saving moves the effective crossover from $(64, 128)$ to $(32, 64)$; the 5% threshold gives the same interval, and the 10% threshold moves it further to $(16, 32)$.

The crossover is therefore better understood as a robustness region than as a single point.

B. Sensitivity to X.509 chain profile

A complementary experiment fixes the Merkle population at $N = 16$ and asks the inverse question: how does changing the X.509 trust path alter the byte advantage?

The same TLS 1.3 configuration is used throughout.

The MTC credential is held fixed while the X.509 baseline changes across direct-server, CA-server, and root-intermediate-server.

Table III reports the first tested X.509 baseline for which MTC becomes byte-efficient.

For ECDSA P-256, MTC is larger than both one-certificate baselines and becomes smaller only when an intermediate certificate is transmitted.

For P-384, MTC is already smaller under the CA-server baseline.

For P-521, it is smaller even under direct-server.

All evaluated ML-DSA and SLH-DSA profiles are already byte-efficient at direct-server.

Table IV gives the full byte-level sensitivity.

The classical profiles illustrate how narrow some transitions are: P-256 is 51 bytes larger than direct-server and only one byte larger than CA-server at $N = 16$, while P-384 changes from a 19-byte disadvantage under direct-server to a 30-byte advantage under CA-server.

P-521 is already 19 bytes smaller than direct-server at the same tree size.

This experiment complements the population sweep rather than duplicating it.

The population analysis increases Merkle proof depth against a fixed X.509 baseline; the chain-profile experiment holds $N = 16$ constant and changes the X.509 authentication path.

Together, they show that the crossover is governed jointly by Merkle population, certificate-path representation, and algorithm-dependent credential size.

V. Why Post-Quantum Profiles Behave Differently

The post-quantum profiles show substantially larger margins than ECDSA.

Even at the largest implemented population, $N = 1024$, every ML-DSA and SLH-DSA landmark credential remains smaller than the corresponding direct-server X.509 baseline.

Table V reports this most conservative one-certificate comparison.

The margin becomes still larger when an intermediate certificate is transmitted.

At $N = 1024$, the root-intermediate savings are 76.48%, 77.20%, and 78.88% for ML-DSA-44, -65, and -87, and 96.26%, 98.10%, and 98.93% for SLH-DSA-128s, -192s, and -256s, respectively.

These margins explain why no post-quantum crossover is observed within the implemented population range.

The result should not be generalized to the complete TLS handshake.

Large post-quantum `CertificateVerify` signatures remain present and are outside the crossover metric.

The narrower conclusion is that, under the evaluated MTC landmark construction and pre-distributed trust model, the credential representation scales substantially more favorably than repeatedly transmitting large X.509 certificates.

VI. Evidence Range and Claim Boundaries

The final crossover claim is intentionally limited to the range exercised by the implementation.

All reported population crossovers are derived from real TLS 1.3 measurements using generated MTC credentials for tested populations up to $N = 1024$.

When the largest tested point remains favorable, we report "> 1024" rather than extrapolating an exact population at which a future crossover might occur.

TABLE I
MEASURED SERVER-ONLY CROSSOVER INTERVALS FOR LANDMARK MTC VERSUS REAL TLS X.509 BASELINES. AN INTERVAL (a, b) MEANS THAT MTC IS SMALLER AT TESTED POPULATION a AND IS NO LONGER SMALLER AT THE NEXT TESTED POPULATION b. "> 1024" MEANS MTC REMAINS SMALLER AT THE LARGEST IMPLEMENTED POPULATION.

Profile	Direct-server	CA-server	Root-intermediate-server
ECDSA P-256	$(4, 8)$	$(8, 16)$	> 1024
ECDSA P-384	$(8, 16)$	$(16, 32)$	> 1024
ECDSA P-521	$(16, 32)$	$(64, 128)$	> 1024
ML-DSA-44	> 1024	> 1024	> 1024
ML-DSA-65	> 1024	> 1024	> 1024
ML-DSA-87	> 1024	> 1024	> 1024
SLH-DSA-128s	> 1024	> 1024	> 1024
SLH-DSA-192s	> 1024	> 1024	> 1024
SLH-DSA-256s	> 1024	> 1024	> 1024

TABLE II
SENSITIVITY OF THE ECDSA CROSSOVER TO A MINIMUM RELATIVE BYTE SAVING. "RAW" CORRESPONDS TO ANY STRICTLY POSITIVE SAVING.

Profile	X.509 baseline	Raw	1%	5%	10%
P-256	Direct	$(4, 8)$	$(4, 8)$	$(1, 4)$	$(1, 4)$
P-256	CA	$(8, 16)$	$(8, 16)$	$(8, 16)$	$(4, 8)$
P-256	Root+Int.	> 1024	> 1024	> 1024	> 1024
P-384	Direct	$(8, 16)$	$(8, 16)$	$(4, 8)$	$(1, 4)$
P-384	CA	$(16, 32)$	$(16, 32)$	$(16, 32)$	$(8, 16)$
P-384	Root+Int.	> 1024	> 1024	> 1024	> 1024
P-521	Direct	$(16, 32)$	$(16, 32)$	$(4, 16)$	$(4, 16)$
P-521	CA	$(64, 128)$	$(32, 64)$	$(32, 64)$	$(16, 32)$
P-521	Root+Int.	> 1024	> 1024	> 1024	> 1024

TABLE III
FIRST TESTED X.509 CHAIN PROFILE AT WHICH $N = 16$ LANDMARK MTC BECOMES BYTE-EFFICIENT. POSITIVE SAVINGS INDICATE FEWER TLS CERTIFICATE-MESSAGE BYTES WITH MTC.

Profile	First favorable baseline	Saving
ECDSA P-256	Root+Int.	+416 B (+47.49%)
ECDSA P-384	CA	+30 B (+5.78%)
ECDSA P-521	Direct	+19 B (+3.48%)
ML-DSA-44	Direct	+2303 B (+57.49%)
ML-DSA-65	Direct	+3192 B (+57.67%)
ML-DSA-87	Direct	+4510 B (+60.19%)
SLH-DSA-128s	Direct	+7739 B (+94.86%)
SLH-DSA-192s	Direct	+16107 B (+97.37%)
SLH-DSA-256s	Direct	+29675 B (+98.50%)

This is a deliberate change from an analytical maximum-range interpretation.

The measured data support statements about the tested population grid and the intervals that it brackets; they do not establish interoperability or byte crossovers for larger trees.

Larger-population serialization models may still be useful as exploratory tools, but they are not part of the primary evidence reported here.

The interval notation also exposes the resolution of the experiment.

For example, P-521/direct-server changes from favorable at $N = 16$ to unfavorable at $N = 32$, while the 5% effective threshold is only bracketed by $N = 4$ and $N = 16$ because no $N = 8$ measurement was required for that profile.

The study therefore reports what was observed, not a more precise value inferred from unmeasured points.

VII. PROTOCOL VALIDATION BEFORE SCALING

The crossover analysis is meaningful only if the underlying MTC objects and TLS integration are themselves validated.

A separate protocol-validation campaign exercised nine signature profiles, 16 leaf identities per profile, and both landmark and standalone MTC modes, producing 288 TLS 1.3 handshakes.

The validation pipeline checked capture integrity and preservation, object parsing and Merkle proof validation, exact TLS transcript reconstruction, independent `CertificateVerify` verification, independent server and client `Finished` verification, and final implemented-subset protocol reports.

All 288 handshakes passed these validation layers.

This protocol-validation campaign is deliberately distinct from the 59-cell population sweep.

The former provides deep protocol evidence at $N = 16$; the latter measures real TLS `Certificate`-message size across multiple generated tree sizes up to $N = 1024$.

Thus, larger-N crossover points are not merely synthetic serialization values, but they have not each been subjected to the same full transcript-validation battery as the dedicated $N = 16$ protocol campaign.

VIII. WHAT THE RESULTS TELL NETWORK DESIGNERS

The measurements reveal three practical regimes.

A. Compact classical authentication

With one transmitted ECDSA certificate, the crossover occurs at small populations.

TABLE IV
BYTE-LEVEL SAVINGS OF $N = 16$ LANDMARK MTC RELATIVE TO THE THREE MEASURED X.509 BASELINES. POSITIVE VALUES INDICATE FEWER TLS CERTIFICATE-MESSAGE BYTES WITH MTC.

Profile	MTC bytes	Direct-server	CA-server	Root-intermediate-server
ECDSA P-256	460	-51 B (-12.47%)	-1 B (-0.22%)	+416 B (+47.49%)
ECDSA P-384	489	-19 B (-4.04%)	+30 B (+5.78%)	+509 B (+51.00%)
ECDSA P-521	527	+19 B (+3.48%)	+66 B (+11.13%)	+619 B (+54.01%)
ML-DSA-44	1703	+2303 B (+57.49%)	+2351 B (+57.99%)	+6364 B (+78.89%)
ML-DSA-65	2343	+3192 B (+57.67%)	+3240 B (+58.03%)	+8782 B (+78.94%)
ML-DSA-87	2983	+4510 B (+60.19%)	+4558 B (+60.44%)	+12058 B (+80.17%)
SLH-DSA-128s	419	+7739 B (+94.86%)	+7787 B (+94.89%)	+15952 B (+97.44%)
SLH-DSA-192s	435	+16107 B (+97.37%)	+16155 B (+97.38%)	+32704 B (+98.69%)
SLH-DSA-256s	451	+29675 B (+98.50%)	+29723 B (+98.51%)	+59856 B (+99.25%)

TABLE V
DIRECT-SERVER X.509 SIZE VERSUS MEASURED LANDMARK MTC SIZE AT $N = 1024$.

Profile	X.509 (B)	MTC (B)	Saving (%)
ML-DSA-44	4006	1897	52.65
ML-DSA-65	5535	2537	54.16
ML-DSA-87	7493	3177	57.60
SLH-DSA-128s	8158	613	92.49
SLH-DSA-192s	16542	629	96.20
SLH-DSA-256s	30126	645	97.86

The direct-server intervals are $(4, 8)$ for P-256, $(8, 16)$ for P-384, and $(16, 32)$ for P-521.

In this regime, an MTC deployment should not be justified by certificate bytes alone, and the effective-threshold analysis matters because the raw advantage can be only a few bytes.

B. Classical authentication with a larger transmitted path

Adding an intermediate certificate raises the X.509 baseline without changing the MTC tree size.

All three ECDSA profiles remain smaller through $N = 1024$ under the root-intermediate-server baseline, with relative savings at $N = 1024$ of 25.34%, 31.56%, and 37.09% for P-256, P-384, and P-521.

The important design variable is therefore not nominal key family alone but the amount of authentication material actually transmitted on the wire.

C. Post-quantum authentication

The post-quantum profiles occupy a different regime.

At $N = 1024$, even the direct-server comparison still favors MTC by 52.65–57.60% for ML-DSA and 92.49–97.86% for SLH-DSA.

For the tested range, Merkle proof growth is too small to erase the much larger X.509 credential baseline.

The central lesson is therefore more precise than "Merkle is smaller":

> *The usefulness of Merkle-based authentication depends jointly on credential family, transmitted X.509 path, Merkle population, and the minimum saving considered operationally meaningful.*

IX. LIMITS OF THE CURRENT STUDY

Several limitations constrain the interpretation of these results.

First, X.509 and MTC credentials are separate issuance fixtures rather than byte-identical transformations of one certificate object.

Their cryptographic profiles and experimental roles are aligned, but representation-specific fields necessarily differ.

Second, the primary crossover is measured only at the discrete tree sizes listed in Section III.

For this reason, ECDSA crossovers are reported as intervals rather than exact populations.

No claim is made about the unmeasured integer values inside those intervals.

Third, the dedicated protocol-conformance campaign performs its deepest transcript and cryptographic validation at $N = 16$.

The crossover campaign executes generated larger-N credentials in real TLS handshakes, but those cells are not all reprocessed through the full independent validation stack.

Fourth, the current crossover uses leaf index zero and the evaluated landmark policy.

Different leaf positions or landmark-selection policies can change the exact inclusion proof and therefore shift the byte boundary.

Fifth, $N = 1024$ is the largest implemented population used for the final crossover evidence.

Configurations reported as "> 1024" are favorable at that boundary; the experiment does not identify where, or whether, they cross beyond it.

Sixth, MTC trust material is pre-distributed.

The per-connection comparison therefore measures TLS authentication traffic, not the external cost of provisioning or updating trust state.

Finally, `CertificateVerify` size, total handshake bytes, CPU cost, packet loss, and bandwidth constraints are outside the primary server-only `Certificate`-message crossover metric.

Latency, connection-establishment timing, bounded Retry, and mutual authentication are evaluated separately as complementary systems experiments.

The mutual-authentication crossover extends the same certificate-representation idea bidirectionally by summing the server and client TLS `Certificate` messages. It covers

ECDSA P-256, P-384, and P-521 against direct and CA X.509 baselines, but it does not yet include post-quantum mutual-authentication crossover curves or a root-intermediate mutual-authentication baseline. Comparisons with the server-only intervals are therefore descriptive sensitivity results across authentication regimes rather than evidence that the two measurement settings are interchangeable.

X. Performance Context: From Authentication Size to Connection Establishment

The certificate-size results presented above characterize the communication cost of the authentication representation itself. A smaller TLS `Certificate` message, however, does not necessarily translate into a proportionally faster connection. End-to-end establishment time also depends on network round-trip time (RTT), transport behavior, cryptographic processing, message fragmentation, and the remaining handshake messages, particularly `CertificateVerify`. We therefore treat representation size and connection performance as complementary measurements.

A. Controlled RTT experiment

To isolate the effect of path latency, we performed a controlled TLS 1.3 experiment comparing PQ X.509, Compact Merkle, and landmark MTC authentication. The cryptographic and protocol configuration was held fixed while only the additional RTT was varied. Six RTT targets were evaluated: 0, 10, 20, 50, 100, and 200 ms. Five independent campaigns were executed per RTT target, and campaign-level medians were used as the statistical unit.

The results show that path latency rapidly becomes the dominant component of handshake time. PQ X.509 increased from a median of 1.463 ms without added latency to 210.803 ms at the 200 ms target. Landmark MTC followed the same trend, from 1.414 ms to 210.923 ms. Compact Merkle was slightly more expensive in several conditions despite transmitting fewer authentication bytes.

The descriptive linear fits reinforce this interpretation. The fitted slopes were 1.0285 ms/ms for PQ X.509, 1.0426 ms/ms for Compact warm, and 1.0295 ms/ms for landmark MTC, with coefficients of determination of 0.99853, 0.99886, and 0.99879, respectively. Thus, the compared authentication mechanisms preserve essentially the same TLS flight structure under the evaluated latency-only conditions.

Landmark MTC therefore does not exhibit a systematic latency advantage over PQ X.509 in the unconstrained-bandwidth experiment. The paired differences remain small relative to the RTT-induced increase. At the 200 ms target, for example, the paired MTC–X.509 effect was -0.364 ms with a bootstrap interval of $[-1.191, 0.988]$ ms, while Compact–X.509 was 2.285 ms with an interval of $[0.941, 2.956]$ ms.

This result is important when interpreting the crossover analysis. The main benefit identified in this study is a reduction in the size and scaling rate of the TLS authentication representation, not an unconditional reduction in handshake latency. Under links with ample bandwidth, RTT dominates the handshake and can mask differences of several kilobytes in authentication material.

B. Independent validation of connection timing

To strengthen the timing methodology, we additionally validated the native client measurements against an independent packet-capture view. Native instrumentation remains the primary performance measurement, while PCAP is used as an on-wire observation and validation source. The two sources are therefore compared but are not merged or corrected against one another.

Three native intervals are retained:

- `tcp_connect_wall_ms_native`: from entry into the native connection operation until successful TCP connection return;
- `handshake_wall_ms`: from `SSL_CB_HANDSHAKE_START` to `SSL_CB_HANDSHAKE_DONE`; and
- `connection\_establishment\_wall\_ms\_native`: from entry into the native connection operation to `SSL_CB_HANDSHAKE_DONE`.

The corresponding PCAP intervals are:

- `tcp_connect_wall_ms`: first SYN to TCP establishment;
- `tls_from_pcap_wall_ms`: ClientHello to client Finished; and

TABLE VI
HANDSHAKE LATENCY UNDER CONTROLLED RTT. VALUES ARE CAMPAIGN-LEVEL MEDIAN NATIVE TLS HANDSHAKE WALL TIMES. THE EXPERIMENT USED TLS 1.3, X25519, TLS_AES_256_GCM_SHA384, SERVER-ONLY AUTHENTICATION, NO SESSION RESUMPTION, NO PACKET LOSS, AND NO BANDWIDTH LIMITATION.

RTT target (ms)	Measured RTT median (ms)	PQ X.509 wall time (ms)	Compact warm wall time (ms)	MTC landmark wall time (ms)
0	0.099	1.463	2.259	1.414
10	11.500	12.614	13.538	12.363
20	22.350	24.720	25.918	24.598
50	53.750	62.788	62.870	62.071
100	103.000	112.208	114.628	111.695
200	204.500	210.803	214.325	210.923

- `connection_handshake_wall_ms`: first SYN to client
  Finished.

The validation campaign used one fixed PQ X.509 authentication scenario
under three path conditions: local, a 100-ms additional-latency condition,
and a constrained condition configured with 100 ms additional RTT,
5 Mbit/s bandwidth, and 1% independent per-egress packet loss. Each
condition contained five warmup connections and 30 measured connections.

The PCAP-enabled observations form a separate instrumentation-validation
population and are not merged with the uncaptured performance campaigns.

All 30 measured connections were pairable in the local and latency-100
conditions. The constrained condition produced 30 successful TLS
connections, of which 29 yielded complete Native/PCAP timing pairs.

Table VII reports the absolute median timing
values over the successful paired observations. The full connection interval
is directly measured in both domains and is not reconstructed as TCP plus
TLS. The different Native and PCAP boundaries also mean that their absolute
values are not expected to be numerically identical.

The absolute timings expose two effects that are hidden when only the
TLS-handshake interval is considered. First, the latency-100 condition adds
approximately one RTT to TCP establishment and another RTT-scale component
to the subsequent TLS exchange, resulting in a median native full
connection-establishment time of 211.733 ms. Second, under the constrained
profile, the native TLS median increases from 107.110 ms to 118.226 ms and
the native full-connection median from 211.733 ms to 222.935 ms, showing the
additional effect of bandwidth limitation and packet loss on top of the
100-ms latency condition.

Table VIII reports the paired Native−PCAP differences.

These are *medians of per-observation paired differences*; they are not
obtained by subtracting the marginal medians in
Table VII. Confidence intervals use 10,000
bootstrap resamples of the paired observations within each condition.

Positive values indicate a larger Native interval.

The paired results show small, consistently positive Native-minus-PCAP
offsets under all three conditions. These offsets are interpreted as
boundary and instrumentation differences rather than as errors to be
corrected. No correction factor is derived, no Native timing is
retrospectively adjusted from PCAP, and no equality or equivalence claim is
made between the two measurement domains.

The validation therefore supports the methodological separation used in
this study: Native timing, and particularly `handshake_wall_ms`,
remains the primary scientific performance measurement, whereas PCAP
provides an independent on-wire observation for TCP/TLS timing, packet
counts, byte counts, retransmissions, and capture-level validation.

C. Mutual authentication

The crossover experiments use server-only authentication to isolate the server credential.

We separately evaluate mutual TLS authentication to determine whether the communication behavior observed at a fixed tree size remains plausible when both peers authenticate.

The implementation was first subjected to functional validation.

Five positive combinations—X.509/X.509, PQ/PQ, MTC/X.509, X.509/MTC, and MTC/MTC—and three negative controls covering missing or untrusted client

TABLE VII
ABSOLUTE MEDIAN NATIVE AND PCAP TIMINGS IN THE INDEPENDENT CONNECTION-TIMING VALIDATION. VALUES ARE COMPUTED OVER SUCCESSFUL PAIRED OBSERVATIONS ONLY. FULL CONNECTION TIME IS DIRECTLY MEASURED AND IS NOT THE SUM OF THE TCP AND TLS COLUMNS.

	Native median (ms)			PCAP median (ms)		
Condition	**TCP**	**TLS**	**Full**	**TCP**	**TLS**	**Full**
Local	0.842	1.861	2.915	0.057	1.581	1.973
Latency-100	104.881	107.110	211.733	103.570	105.549	209.580
Constrained	104.473	118.226	222.935	103.483	116.719	220.468

TABLE VIII
MEDIAN PAIRED NATIVE−PCAP TIMING DIFFERENCES WITH 95% BOOTSTRAP CONFIDENCE INTERVALS. DIFFERENCES ARE COMPUTED PER PAIRED OBSERVATION BEFORE TAKING THE MEDIAN.

Condition	**TCP delta (ms)**	**TLS delta (ms)**	**Full delta (ms)**
Local	0.782 [0.742, 0.898]	0.276 [0.262, 0.301]	0.834 [0.769, 0.940]
Latency-100	0.808 [0.307, 1.569]	1.157 [0.560, 2.139]	2.140 [1.644, 2.797]
Constrained	1.208 [0.680, 1.682]	0.857 [0.582, 1.332]	2.214 [1.887, 3.219]

credentials all behaved as expected, yielding an 8/8 preflight result.

The quantitative campaign then executed 30 measured rounds for each positive combination, for 150 measured mutual-authentication connections.

TLS 1.3, X25519, TLS_AES_256_GCM_SHA384, disabled tickets, and disabled session resumption were held fixed.

The MTC mutual-authentication fixtures use landmark credentials at $N = 16$; consequently, this experiment is a fixed-point systems comparison, not a mutual-authentication population crossover.

The primary outcome is bidirectional communication volume. Host-side wall-clock measurements are retained only as auxiliary systems context because they include process, Docker, and client/server orchestration overhead.

MTC-specific setup and verification times are reported separately from that host-side wall clock.

The dominant signal is the communication footprint.

Mutual PQ X.509 authentication requires a median of 18,767 transport bytes, compared with 2,029 bytes for the classical X.509/X.509 baseline.

This is a $9.249\times$ increase, or 16,738 additional bytes per connection.

The three fixed-$N = 16$ configurations involving MTC remain close to the classical baseline.

MTC/X.509 requires 2,057 bytes, X.509/MTC 2,050 bytes, and MTC/MTC 2,064 bytes—only 28, 21, and 35 bytes above X.509/X.509, respectively.

Thus, at the evaluated tree size, placing MTC authentication on either or both sides does not reproduce the large bidirectional expansion observed for PQ X.509.

The MTC-specific timing instrumentation provides local-processing context.

With MTC only on the server side, median setup and verification costs are 0.590 ms and 0.111 ms.

With MTC only on the client-authentication side, they are 0.672 ms and 0.109 ms.

With MTC at both endpoints, the available endpoint observations aggregate to 1.209 ms of setup and 0.183 ms of verification.

These values should not be compared directly with the host-side wall clock as if they represented the same timing boundary.

Most importantly, this fixed-point result must not be read as evidence that the server-only crossover intervals remain unchanged under mutual authentication.

Once both peers carry population-dependent MTC credentials, the byte balance can shift.

This observation motivated the dedicated bidirectional population sweep reported next.

D. Bidirectional mutual-authentication crossover

We next evaluate a controlled mutual-authentication crossover using ECDSA P-256, P-384, and P-521 at both endpoints. The experiment preserves TLS 1.3, X25519, `TLS_AES_256_GCM_SHA384`, disabled tickets, and disabled session resumption. For X.509, each curve is evaluated against two bidirectional comparator families: a direct baseline and a CA baseline. For MTC, both endpoints use landmark credentials at the same tree size. The tested populations are $N \in \{1, 4, 8, 16, 32\}$ for P-256, $N \in \{1, 4, 8, 16, 32, 64\}$ for P-384, and $N \in \{4, 8, 16, 32, 64, 128\}$ for P-521.

The final matrix contains six X.509 baselines and 17 MTC cells, for 23 cases in total. Each case is included in five warmup rounds and 30 measured rounds, with case order shuffled within each round. The campaign consequently contains 115 warmup handshakes and 690 measured handshakes. All 690 measured observations passed the certificate-direction invariants used to verify that the server and client views of each transmitted `Certificate` message agree.

The primary metric is

$$B_{\text{mutual}} = B_{\text{Certificate,server}} + B_{\text{Certificate,client}}, \qquad (4)$$

that is, the sum of the complete TLS `Certificate` handshake messages transmitted in both directions. `CertificateVerify` bytes and complete handshake transport volume are retained only as secondary diagnostics

TABLE IX
MUTUAL-AUTHENTICATION RESULTS OVER 30 MEASURED ROUNDS PER CONFIGURATION. COMMUNICATION BYTES ARE THE PRIMARY COMPARISON. HOST-SIDE WALL TIME INCLUDES PROCESS, DOCKER, AND CLIENT/SERVER ORCHESTRATION OVERHEAD AND IS THEREFORE AUXILIARY.

Server/client	Wall (ms)	Total bytes	Byte ratio vs. X.509	MTC setup (ms)	MTC verify (ms)
X.509 / X.509	214.116	2029	1.000	–	–
PQ / PQ	210.862	18767	9.249	–	–
MTC / X.509	212.897	2057	1.014	0.590	0.111
X.509 / MTC	213.557	2050	1.010	0.672	0.109
MTC / MTC	214.881	2064	1.017	1.209	0.183

and do not define the crossover. This keeps the mutual-authentication study aligned with the representation-size question used in the server-only crossover while extending it to both authenticated peers.

Table X reports the raw bidirectional certificate crossovers and the byte margin at the last tested favorable population.

The raw intervals are therefore $(1, 4)$, $(4, 8)$, and $(16, 32)$ for the direct baselines, and $(8, 16)$, $(16, 32)$, and $(32, 64)$ for the CA baselines, for P-256, P-384, and P-521 respectively. Several transition margins are very small. P-256/direct becomes unfavorable at $N = 4$ by only 2 bytes, P-384/direct becomes unfavorable at $N = 8$ by only 3 bytes, and P-521/CA becomes unfavorable at $N = 64$ by only 4 bytes. The last favorable P-521/direct point is also marginal: at $N = 16$, MTC saves only 9 bytes, or 0.823%. These small margins make a thresholded interpretation necessary rather than optional.

Table XI reports the effective crossover for minimum savings of 1%, 5%, and 10%.

The newly measured P-521 point at $N = 8$ is particularly useful for resolving the thresholded direct-baseline region. At that population, the bidirectional MTC certificate total is 1,021 bytes versus 1,094 bytes for X.509/direct, a saving of 73 bytes or 6.673%. Consequently, the P-521/direct effective interval is $(8, 16)$ for both the 1% and 5% thresholds and $(4, 8)$ for the 10% threshold, while the raw interval remains $(16, 32)$. For P-256/direct, no tested population reaches a 10% saving; even $N = 1$ saves only 7.552%.

Compared descriptively with the server-only crossover, mutual authentication does not produce a uniform shift. For the direct baseline, P-256 changes from $(4, 8)$ to $(1, 4)$ and P-384 from $(8, 16)$ to $(4, 8)$, while P-521 remains at $(16, 32)$. For the CA baseline, P-256 remains at $(8, 16)$ and P-384 at $(16, 32)$, whereas P-521 changes from $(64, 128)$ to $(32, 64)$. Because the mutual-authentication metric sums separately issued client and server credentials while the server-only metric contains only the server credential, these interval changes should be read as sensitivity to authentication regime, not as a simple doubling law or a causal shift attributable to one isolated byte component.

E. Bounded trust-anchor Retry

The connection-level evaluation was further extended with a bounded Retry mechanism for trust-anchor negotiation. The objective of this experiment is not to compare authentication schemes, but to quantify the additional connection-establishment cost introduced when the initially advertised trust anchor cannot be used and the client must establish a second TLS connection with a compatible anchor identifier.

The evaluated mechanism permits exactly one Retry. In the normal *happy-path* case, the client advertises a trust-anchor identifier that the server can satisfy and authentication completes using a single connection. In the Retry case, the first connection fails because the advertised trust anchor is incompatible with the server credential. The server exposes the compatible identifier, after which the client starts a new TLS connection using that identifier. The second connection must authenticate successfully. The two attempts use distinct connections and session resumption is disabled.

The expected state transitions are therefore:

$$\begin{aligned} \text{Happy path:} \quad & A \rightarrow \text{PASS}, \\ \text{Retry path:} \quad & A \rightarrow \text{FAIL} \rightarrow B \rightarrow \text{PASS}. \end{aligned} \tag{5}$$

No third connection is permitted. Consequently, the Retry mechanism is strictly bounded to one additional connection and cannot enter an uncontrolled negotiation loop.

The full Retry campaign contained five warmups and 30 measured observations for each path, resulting in 10 warmups and 60 measured observations. Measured observations were paired by round, producing 30 Happy–Retry pairs.

The protocol configuration remained TLS 1.3 with X25519, TLS_AES_256_GCM_SHA384, server-only authentication, no tickets and no session resumption.

Table XII reports the median native connection timings. The Happy path requires one TCP connection and one TLS handshake, whereas the Retry path includes the complete cost of the failed first connection and the successful second connection.

Because the observations were generated in paired rounds, the primary comparison uses per-round Retry-minus-Happy differences rather than the difference between marginal medians. The median paired overheads were 0.409 ms for TCP establishment, 0.472 ms for TLS, and 0.898 ms for complete native connection establishment. The corresponding 95% bootstrap intervals were $[0.372, 0.433]$ ms, $[0.435, 0.496]$ ms, and $[0.872, 0.935]$ ms, respectively.

The Retry path therefore increases connection-establishment time, as expected, because an additional connection is required. However, under the local conditions used for this validation, the measured median overhead remains below 1 ms for the complete native connection-establishment interval. This result should not be interpreted as a general network-cost

TABLE X
BIDIRECTIONAL MUTUAL-AUTHENTICATION CROSSOVER USING THE SUM OF SERVER AND CLIENT TLS CERTIFICATE MESSAGES. POSITIVE ΔB MEANS MTC TRANSMITS FEWER CERTIFICATE-MESSAGE BYTES THAN THE CORRESPONDING BIDIRECTIONAL X.509 BASELINE.

Profile	X.509 baseline	X.509 (B)	Last favorable N	MTC (B)	ΔB (B)	Raw interval
P-256	Direct	821	1	759	+62	$(1, 4)$
P-256	CA	942	8	887	+55	$(8, 16)$
P-384	Direct	942	4	881	+61	$(4, 8)$
P-384	CA	1062	16	1009	+53	$(16, 32)$
P-521	Direct	1094	16	1085	+9	$(16, 32)$
P-521	CA	1211	32	1151	+60	$(32, 64)$

TABLE XI
SENSITIVITY OF THE BIDIRECTIONAL MUTUAL-AUTHENTICATION CROSSOVER TO A MINIMUM RELATIVE CERTIFICATE-MESSAGE SAVING. "RAW" REQUIRES ANY STRICTLY POSITIVE BYTE SAVING. "NONE" MEANS THAT NO TESTED POPULATION MEETS THE REQUESTED THRESHOLD.

Profile	X.509 baseline	Raw	1%	5%	10%
P-256	Direct	$(1, 4)$	$(1, 4)$	$(1, 4)$	none
P-256	CA	$(8, 16)$	$(8, 16)$	$(8, 16)$	$(4, 8)$
P-384	Direct	$(4, 8)$	$(4, 8)$	$(4, 8)$	$(1, 4)$
P-384	CA	$(16, 32)$	$(16, 32)$	$(8, 16)$	$(8, 16)$
P-521	Direct	$(16, 32)$	$(8, 16)$	$(8, 16)$	$(4, 8)$
P-521	CA	$(32, 64)$	$(32, 64)$	$(16, 32)$	$(16, 32)$

TABLE XII
NATIVE CONNECTION TIMING FOR THE BOUNDED RETRY EXPERIMENT. RETRY VALUES INCLUDE BOTH CONNECTION ATTEMPTS.

Path	TCP (ms)	TLS (ms)	Full (ms)
Happy	0.510	0.864	1.410
Retry	0.914	1.328	2.297

TABLE XIII
PAIRED RETRY-MINUS-HAPPY OVERHEAD. CONFIDENCE INTERVALS ARE 95% BOOTSTRAP INTERVALS FOR THE MEDIAN PAIRED DIFFERENCE OVER 30 ROUNDS.

Interval	Median Δ (ms)	95% CI (ms)
TCP	0.409	[0.372, 0.433]
TLS	0.472	[0.435, 0.496]
Full	0.898	[0.872, 0.935]

bound: under non-negligible RTT, the additional connection necessarily introduces additional path-delay cost.

The experiment also validates an important protocol property independently of its timing cost. Retry remains bounded: the successful Retry observations use exactly two distinct connections, one failed and one authenticated, with a retry count of one and no session reuse. Failure cases in which the second trust anchor is also unsuitable terminate after the second attempt rather than initiating a third connection.

This mechanism is therefore complementary to the certificate-size crossover analysis. The crossover results quantify the scaling of the authentication representation, whereas the Retry experiment quantifies the connection-level cost of dynamically recovering from an initially incompatible trust-anchor selection.

F. Implications for the crossover result

The systems experiments clarify, but do not redefine, the server-only crossover metric.

Under high-bandwidth, low-loss paths, a substantial reduction in authentication bytes can produce little visible change in handshake latency because RTT dominates the exchange.

Conversely, large authentication objects become more consequential when serialization, fragmentation, bandwidth limitation, or retransmission cost matters.

Certificate-size scalability and connection-establishment performance should therefore be treated as complementary metrics:

$$\begin{aligned} &\text{smaller authentication bytes} \\ &\quad \not\Rightarrow \text{ proportionally lower connection time.} \end{aligned} \tag{6}$$

The server-only crossover answers a representation question:

for a tested X.509 baseline, over what measured population range does landmark MTC require fewer TLS `Certificate`-message bytes?

The RTT and Native–PCAP experiments answer a different systems question:

under which path conditions does that byte reduction become visible in TLS or complete connection establishment?

Mutual authentication introduces a third axis.

The fixed-$N = 16$ systems comparison shows that MTC-based bidirectional authentication can remain close to the classical transport footprint, while the dedicated certificate-size crossover shows that authenticating both peers can change the observed population boundary in a profile-dependent way.

Across the ECDSA direct baselines, the mutual-authentication raw intervals are $(1, 4)$, $(4, 8)$, and $(16, 32)$ for P-256, P-384, and P-521; across the CA baselines they are $(8, 16)$, $(16, 32)$, and $(32, 64)$.

The comparison with server-only authentication is deliberately descriptive: the mutual metric sums separately issued client and server `Certificate` messages, so the resulting interval shifts should not be interpreted as a universal multiplier or as evidence that server-only and mutual-authentication crossovers are interchangeable.

XI. Where to Go Next

The mutual-authentication crossover now covers the three evaluated ECDSA curves under both direct and CA X.509 baselines, so the next extension should be selective rather than a larger classical matrix.

A useful next step is to add one or two representative post-quantum mutual-authentication profiles and, if needed, a root-intermediate comparator to determine whether the large server-only PQ margins persist when both peers authenticate.

The thresholded metric N_τ^* should remain the primary robustness view, because the present ECDSA results already show that raw mutual-authentication crossovers can be separated from practically meaningful savings by one or more tested tree sizes.

A second direction is to reduce proof transmission through authenticated client state.

Progressive Merkle-state reuse could allow clients to retain verified tree nodes while servers transmit only missing proof components.

Such mechanisms move the design problem from static credential size toward an explicit storage–communication trade-off.

A third direction is to compare MTC with raw public keys and hybrid designs that combine raw-key authentication with Merkle membership.

This would help separate the cost of PKI semantics from the cost of public-key representation.

Finally, the latency-only and Native–PCAP validation experiments should be complemented by bandwidth-limited, packet-loss, and selected combined-path conditions.

Those experiments can establish when the byte reductions measured by the crossover campaign translate into observable improvements in TLS or full connection establishment.

XII. Conclusion

Post-quantum authentication makes certificate representation a first-order TLS design concern again.

The final measurements show that MTC and X.509 scale according to different rules, but they also show that the crossover must be stated with care.

In the unified 59-cell server-only campaign, the measured direct-server ECDSA crossover intervals are $(4, 8)$ for P-256, $(8, 16)$ for P-384, and $(16, 32)$ for P-521.

Under the CA-server baseline they move to $(8, 16)$, $(16, 32)$, and $(64, 128)$.

With a transmitted intermediate certificate, all three ECDSA profiles remain smaller through $N = 1024$.

Every evaluated ML-DSA and SLH-DSA profile likewise remains favorable through $N = 1024$ for all three X.509 baselines.

The thresholded analysis adds an important qualification.

A raw crossover can be driven by a negligible byte difference: P-521/CA-server is still favorable at $N = 64$ by only one byte.

Requiring a 1% or 5% saving moves its effective crossover to $(32, 64)$, and requiring 10% moves it to $(16, 32)$.

Reporting N_τ^* alongside the raw crossover therefore gives a more robust picture of when MTC remains materially advantageous.

The complementary chain-profile experiment reaches the same conclusion from the opposite direction: increasing transmitted X.509 path material rapidly favors MTC, while the evaluated post-quantum profiles are already byte-efficient at the shallowest baseline.

At $N = 1024$, direct-server savings remain 52.65–57.60% for ML-DSA and 92.49–97.86% for SLH-DSA.

These representation gains do not imply an unconditional latency gain.

The controlled RTT and Native–PCAP experiments show that network delay can dominate connection time, while bounded Retry adds a separate connection-level cost when trust-anchor negotiation requires a second attempt.

The fixed-$N = 16$ mutual-authentication experiment shows that MTC combinations remain close to the classical bidirectional transport footprint, while the dedicated mutual-authentication crossover extends the certificate-size analysis to both authenticated peers.

Across 690 measured handshakes, the raw direct-baseline intervals are $(1, 4)$ for P-256, $(4, 8)$ for P-384, and $(16, 32)$ for P-521; the corresponding CA-baseline intervals are $(8, 16)$, $(16, 32)$, and $(32, 64)$.

The thresholded mutual-authentication analysis again exposes marginal byte advantages: P-521/direct remains raw-favorable through $N = 16$ by only 9 bytes (0.823%), so its 1% and 5% effective intervals move to $(8, 16)$, while P-256/direct never reaches a 10% saving even at $N = 1$.

The resulting design lesson is therefore multidimensional:

credential family, transmitted trust path, Merkle population, and the minimum saving considered meaningful jointly determine whether MTC is byte-efficient.

Keeping those dimensions separate from latency and deployment-state costs makes the crossover result both more reproducible and more useful.

Acknowledgment

To be completed.

References

[1] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018.
[2] IETF, "Merkle Tree Certificates," Internet-Draft, `draft-ietf-plants-merkle-tree-certs-05`.
[3] IETF, "TLS Trust Anchor Identifiers," Internet-Draft, `draft-ietf-tls-trust-anchor-ids-04`.
[4] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard," FIPS 204.
[5] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard," FIPS 205.
[6] OpenSSL Project, "OpenSSL," software documentation and source code.
[7] Open Quantum Safe Project, "liboqs: C library for quantum-resistant cryptographic algorithms."